

BIG DATA PROJECT DOCUMENTATION

Implementasi Pipeline Big Data untuk Deteksi Anomali Transaksi dan Analisis Perilaku Pelanggan pada Online Retail Dataset Menggunakan Apache Spark, PostgreSQL, dan MongoDB



Disusun Oleh:

11423003	:	Samuel Rizki Sinambela
11423010	:	Agatha C. S. Siahaan
11423025	:	Yos Ivan Sirait
11423028	:	Grace Advryanti Tampubolon
11423054	:	Natan Obet Nego Hutahaeen

PROGRAM STUDI TEKNOLOGI REKAYASA PERANGKAT LUNAK

**FAKULTAS VOKASI
INSTITUT TEKNOLOGI DEL
2026**

Daftar Isi

BAB I.....	4
PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Rumusan Masalah.....	4
1.3 Tujuan Proyek.....	5
1.4 Manfaat Proyek.....	5
1.5 Pembagian Task Per Anggota.....	5
BAB II.....	6
TINJAUAN PUSTAKA.....	6
2.1 Karakteristik Big Data.....	6
2.1.1 Volume.....	6
2.1.2 Velocity.....	6
2.1.3 Variety.....	6
2.1.4 Veracity.....	6
2.1.5 Value.....	6
2.2 Teknologi yang Digunakan.....	7
BAB III.....	8
ANALISIS DAN PERANCANGAN SISTEM.....	8
3.1 Sumber Dataset.....	8
3.2 Perancangan Arsitektur Sistem.....	8
3.3 Komponen Teknologi.....	9
3.4 Alur Data Detail.....	9
BAB IV.....	10
IMPLEMENTASI.....	10
4.1 Pipeline Data Cleaning dengan PySpark.....	10
4.1.1 Inisiasi Spark dan Load Data.....	10
4.1.2 Exploratory Data Analysis (EDA).....	11
4.1.3 Penanganan Missing Values.....	13
4.1.4 Penghapusan Duplikat dan Data Tidak Valid.....	14
4.1.5 Laporan Perbandingan Sebelum dan Sesudah Cleaning.....	14
4.2 Feature Engineering dengan PySpark.....	15
4.2.1 Perhitungan Nilai Transaksi.....	15
4.2.2 Pembacaan Dataset Hasil Cleaning.....	16
4.3 Data Transformation.....	23
4.4 Integrasi Database.....	24
4.4.1 PostgreSQL - Database Relasional.....	24
4.4.2 MongoDB - Database Non-Relasional.....	26
4.4.3 Cross-System Query - Verifikasi Konsistensi Data.....	28

BAB V.....	29
HASIL DAN ANALISIS.....	29
5.1 Hasil Data Cleaning.....	29
5.2.1 Pendapatan dan Penjualan (Revenue).....	29
5.2.2 Distribusi Geografis dan Lokasi Risiko.....	30
5.2.3 Analisis Produk Terlaris.....	30
5.2.4 Pola Waktu dan Jam Sibuk.....	30
5.2.5 Ringkasan Anomali.....	31
5.2.6 Korelasi Fitur.....	32
BAB VI.....	33
KESIMPULAN.....	33
6.1 Kesimpulan.....	33
6.2 Saran.....	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan internet telah mendorong pertumbuhan transaksi e-commerce secara eksponensial. Setiap hari, platform retail online menghasilkan jutaan transaksi yang melibatkan ribuan produk dan pelanggan dari berbagai penjuru dunia. Volume data yang besar, kecepatan produksi yang tinggi, serta keberagaman jenis data tersebut menciptakan tantangan serius dalam pengelolaan dan analisis data secara konvensional.

Kondisi ini mendorong adopsi teknologi Big Data sebagai solusi untuk memproses dan menganalisis data dalam skala besar. Salah satu isu kritis dalam industri retail adalah keberadaan transaksi anomali yang dapat mengindikasikan adanya kesalahan sistem, penipuan (fraud), atau perilaku pembelian yang tidak wajar. Deteksi dini terhadap anomali semacam ini sangat penting untuk menjaga integritas data dan melindungi kerugian bisnis.

Selain deteksi anomali, pemahaman mendalam tentang perilaku pelanggan juga menjadi faktor kunci dalam strategi bisnis modern. Segmentasi pelanggan berdasarkan pola pembelian memungkinkan perusahaan untuk merancang strategi pemasaran yang lebih tepat sasaran, meningkatkan retensi pelanggan, dan mengoptimalkan pendapatan.

Proyek ini hadir sebagai solusi komprehensif dengan mengimplementasikan pipeline Big Data end-to-end menggunakan Apache Spark sebagai engine pemrosesan data terdistribusi, PostgreSQL sebagai penyimpanan data relasional terstruktur, dan MongoDB sebagai penyimpanan data non-relasional untuk hasil analisis anomali. Dataset yang digunakan adalah Online Retail Dataset yang berisi ratusan ribu transaksi dari sebuah retail online berbasis di Inggris.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini sebagai berikut:

1. Bagaimana membuat pipeline data retail yang dapat memproses dataset besar dan menjaga kualitas data?
2. Bagaimana menghasilkan fitur yang berguna untuk mendeteksi transaksi anomali?
3. Bagaimana menyimpan hasil pemrosesan ke database yang dapat diakses oleh dashboard?
4. Bagaimana membuat dashboard yang menampilkan metrik anomali, korelasi fitur, dan insight bisnis secara jelas?
5. Apa penyebab perbedaan jumlah anomaly antara hasil awal dan tampilan dashboard?

1.3 Tujuan Proyek

Tujuan dari proyek ini adalah:

1. Membangun pipeline Big Data end-to-end yang mencakup proses ingest, cleaning, transformasi, feature engineering, dan penyimpanan data transaksi retail
2. Mengimplementasikan feature engineering agar model anomaly memiliki variabel yang lebih baik.
3. Mengimplementasikan sistem deteksi anomali transaksi menggunakan algoritma Isolation Forest dengan Apache Spark dan scikit-learn.
4. Mengintegrasikan PostgreSQL sebagai data warehouse relasional dan MongoDB sebagai penyimpanan dokumen untuk hasil analisis anomali.
5. Menyajikan dashboard visual dan insight bisnis yang mudah dipahami sebagai dasar pengambilan keputusan strategis di bidang retail.

1.4 Manfaat Proyek

Proyek ini diharapkan dapat memberikan manfaat dalam memahami penerapan teknologi Big Data dan analisis data pada sistem retail. Adapun manfaat yang diperoleh yaitu:

1. Memahami penggunaan Apache Spark untuk data besar.
2. Mempelajari integrasi antara sistem batch processing dan dashboard analytics.
3. Mengaplikasikan konsep feature engineering dan anomaly detection dalam kasus nyata.
4. Mendokumentasikan arsitektur dan proses ETL untuk laporan akademik.

1.5 Pembagian Task Per Anggota

Tuliskan isi pembahasan pada bagian ini...

NIM	Nama	Tanggung Jawab
11423003	Samuel Rizki Sinambela	
11423010	Agatha C. S. Siahaan	
11423025	Yos Ivan Sirait	
11423028	Grace Advryanti Tampubolon	
11423054	Natan Obet Nego Hutahaeen	

BAB II

TINJAUAN PUSTAKA

2.1 Karakteristik Big Data

Big Data adalah istilah yang merujuk pada kumpulan data dalam jumlah besar yang sangat kompleks sehingga tidak dapat diproses menggunakan sistem basis data tradisional. Karakteristik data besar yang mendasari proyek ini meliputi volume, velocity, variety, veracity, dan value.

2.1.1 Volume

Volume mengacu pada skala ukuran data yang diproses. Proyek ini menggunakan Online Retail Dataset yang mencakup 541.909 baris transaksi awal. Skala data yang besar ini menuntut penggunaan Apache Spark untuk pemrosesan data terdistribusi yang efisien, guna menangani volume yang tinggi.

2.1.2 Velocity

Velocity mencerminkan kecepatan produksi dan aliran data. Data transaksi retail memiliki kecepatan aliran yang tinggi, sehingga teknologi yang digunakan (Apache Spark) harus mendukung arsitektur batch processing yang cepat dan terotomasi. Hal ini penting agar hasil analisis dan deteksi anomali dapat diperbarui secara berkala.

2.1.3 Variety

Variety merujuk pada keberagaman jenis dan format data. Data transaksi retail mengandung kombinasi data numerik (Quantity, UnitPrice), data kategorikal (Country, StockCode), hingga data waktu (InvoiceDate). Feature engineering diperlukan untuk mengubah format yang beragam ini (seperti String dan Categorical) menjadi representasi numerik yang seragam agar dapat diolah oleh model deteksi anomali.

2.1.4 Veracity

Veracity berhubungan dengan keakuratan dan keandalan data. Dalam proyek ini, data mentah mengandung nilai hilang (missing values), duplikat, dan nilai tidak valid (seperti kuantitas negatif). Kondisi ini mengharuskan pelaksanaan proses data cleaning yang ketat untuk menjaga akurasi model Isolation Forest dan mencegah anomali palsu yang disebabkan oleh kualitas data yang buruk.

2.1.5 Value

Value adalah dimensi terpenting dari Big Data yang mengacu pada nilai bisnis dan ekonomis yang dihasilkan dari analisis data. Dalam konteks proyek ini, Value diwujudkan melalui dua hasil utama: Insight transaksi abnormal dan potensi fraud (melalui deteksi anomali) serta rekomendasi operasional strategis yang didapat dari analisis perilaku pelanggan (seperti identifikasi produk terlaris dan pola jam sibuk).

2.2 Teknologi yang Digunakan

1. Apache Spark & PySpark

Apache Spark dan PySpark digunakan sebagai mesin pemrosesan utama untuk membaca, mengolah, dan menulis data dalam jumlah besar secara cepat. Teknologi ini memanfaatkan komputasi terdistribusi sehingga proses pengolahan data dapat dilakukan dengan lebih efisien.

2. Isolation Forest (Scikit-learn)

Isolation Forest merupakan algoritma *machine learning* yang digunakan untuk mendeteksi data anomali. Algoritma ini bekerja dengan mengidentifikasi dan mengisolasi data yang memiliki pola berbeda secara signifikan dibandingkan dengan mayoritas data normal.

3. PostgreSQL

PostgreSQL digunakan sebagai basis data relasional untuk menyimpan data transaksi yang telah terstruktur dengan rapi. Basis data ini dipilih karena memiliki stabilitas yang baik serta mendukung penggunaan bahasa SQL untuk pengelolaan data.

4. MongoDB

MongoDB digunakan sebagai basis data NoSQL berbasis dokumen untuk menyimpan riwayat transaksi anomali. Basis data ini dipilih karena fleksibel dalam menyimpan data dengan struktur yang beragam dan detail yang kompleks.

5. Apache Parquet

Apache Parquet digunakan sebagai format penyimpanan data sementara dalam bentuk file terkompresi. Format ini memungkinkan proses pembacaan dan pemrosesan data dilakukan dengan lebih ringan dan cepat.

6. Streamlit & Plotly

Streamlit dan Plotly digunakan untuk membangun dashboard analitik serta menampilkan visualisasi data secara interaktif. Dengan teknologi ini, hasil analisis dapat disajikan dalam bentuk grafik yang mudah dipahami oleh pengguna.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Sumber Dataset

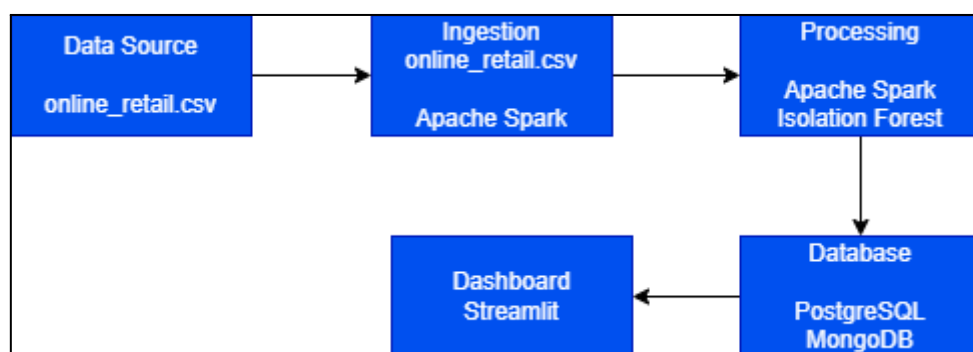
Proyek ini menggunakan kumpulan data riwayat transaksi sebuah retail online berbasis di Inggris. Datanya diambil dari file bernama `online_retail.csv` yang bersumber dari Kaggle.

Kolom	Deskripsi
InvoiceNo	Nomor unik faktur
StockCode	Kode barang
Description	Nama dan deskripsi produk
Quantity	Jumlah volume produk yang dibeli
InvoiceDate	Waktu transaksi (tanggal & jam)
UnitPrice	Harga dari setiap barang per unit
CustomerID	ID unik pengenalan pengguna
Country	Identitas negara asal transaksi
TotalPrice	Akumulasi tagihan dari produk tersebut (dihitung secara mandiri dari Quantity dikalikan UnitPrice)

Pada kondisi aslinya, data mentah ini masih bermasalah. Terdapat baris yang kosong, data yang duplikat, dan ada jumlah barang yang angkanya minus (menandakan barang tersebut dikembalikan atau retur). Selain itu, format penanggalan juga masih berupa teks biasa sehingga tidak bisa dihitung sebagai waktu oleh sistem.

3.2 Perancangan Arsitektur Sistem

Sistem ini dirancang menggunakan konsep arsitektur ETL (Extract, Transform, Load) secara berurutan:



1. **Input / Ingestion**

Membaca data transaksi dari file `online_retail.csv` menggunakan Apache Spark untuk diproses ke tahap berikutnya.

2. **Processing**

Melakukan data cleaning, feature engineering, perhitungan performa pelanggan, serta deteksi anomali menggunakan algoritma Isolation Forest.

3. **Penyimpanan Sementara**

Menyimpan hasil processing ke format Parquet (`cleaned_data`, `featured_data`, `advanced_anomaly`) agar proses pengolahan dan transfer data lebih ringan dan efisien.

4. **Database**

Menyimpan data hasil akhir ke PostgreSQL untuk data terstruktur dan MongoDB untuk hasil laporan anomali.

5. **Dashboard**

Menampilkan hasil analisis dan visualisasi data melalui dashboard interaktif menggunakan Streamlit.

3.3 Komponen Teknologi

Teknologi utama yang melengkapi sistem ini merangkumi:

1. Spark: Memproses data berskala besar secara berkelompok (*batch*).
2. PySpark: API Python yang digunakan untuk berinteraksi dengan Spark.
3. scikit-learn Isolation Forest: Model pembelajaran mesin bagi mengesan anomali.
4. PostgreSQL: Storan pangkalan data hubungan.
5. MongoDB: Storan pangkalan data berasaskan dokumen.
6. Plotly & Streamlit: Pustaka untuk membina visualisasi dan papan muka yang interaktif.

3.4 Alur Data Detail

1. Ingestion: Membaca CSV menggunakan Spark dengan `inferSchema`.
2. Cleaning: Menghapus NULL, duplikat, dan memvalidasi nilai numerik.
3. Transformation: Standarisasi tanggal dan perhitungan `TotalPrice`.
4. Engineering: Ekstraksi fitur waktu dan agregasi perilaku pelanggan.
5. Modeling: Pelabelan anomali (1 untuk anomali, 0 untuk normal).
6. Storage: Ekspor paralel ke PostgreSQL (tabel) dan MongoDB (dokumen).
7. Visualization: Dashboard memuat data untuk analisis tren dan sebaran anomali.

BAB IV

IMPLEMENTASI

4.1 Pipeline Data Cleaning dengan PySpark

Tahap data cleaning merupakan proses penting dalam pipeline Big Data karena kualitas hasil analisis sangat dipengaruhi oleh kualitas data yang digunakan. Pada penelitian ini, proses pembersihan data dilakukan menggunakan Apache Spark melalui library PySpark yang diimplementasikan pada file `cleaning.py`. Tahapan ini bertujuan untuk memastikan data memiliki tingkat akurasi, konsistensi, dan kelengkapan yang baik sebelum digunakan pada proses analisis lanjutan maupun machine learning.

4.1.1 Inisiasi Spark dan Load Data

Tahap pertama dimulai dengan mengaktifkan Apache Spark melalui pembuatan `SparkSession` sebagai inti dari proses pemrosesan data terdistribusi. Setelah Spark berhasil dijalankan, sistem membaca file dataset mentah bernama `online_retail.csv` dan memuatnya ke dalam memori untuk diproses lebih lanjut.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder \
    .appName("RetailCleaning") \
    .getOrCreate()

print("SPARK BERHASIL DIJALANKAN")

df = spark.read.csv(
    "data/online_retail.csv",
    header=True,
    inferSchema=True,
    sep=";"
)
```

Selanjutnya, sistem menampilkan struktur data (schema) untuk memastikan setiap kolom berhasil dikenali dengan benar. Pemeriksaan ini penting untuk memastikan tipe data awal sesuai dengan kebutuhan proses analisis berikutnya. Selain itu, tahap ini juga berfungsi untuk memastikan bahwa file dataset telah berhasil terhubung ke sistem tanpa mengalami kesalahan pembacaan.

```
-----
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
-----
TAHAP 4.1: PIPELINE DATA CLEANING DENGAN PYSARK
-----

[TAHAP 4.1.1] INISIASI SPARK DAN LOAD DATA
-----
✓ Apache Spark berhasil dijalankan
✓ Dataset berhasil dibaca dari: data/online_retail.csv
✓ Jumlah data awal: 541909 baris
```

4.1.2 Exploratory Data Analysis (EDA)

Setelah data berhasil dimuat, sistem melakukan proses Exploratory Data Analysis (EDA) untuk memahami kondisi data secara umum. Pada tahap ini, sistem menampilkan beberapa contoh data menggunakan fungsi `show()` untuk melihat isi dataset yang akan diproses. Sistem juga menampilkan struktur data menggunakan fungsi `printSchema()` untuk memastikan setiap kolom telah terbaca dengan tipe data yang sesuai. Selanjutnya, sistem melakukan pemeriksaan nilai minimum dan maksimum pada kolom `Quantity` dan `UnitPrice`. Pemeriksaan ini bertujuan untuk mendeteksi kemungkinan adanya data yang tidak valid seperti jumlah barang negatif atau harga bernilai nol. Selain itu, sistem melakukan analisis distribusi transaksi berdasarkan negara menggunakan fungsi `groupBy()` pada kolom `Country`. Analisis ini membantu dalam mengetahui persebaran transaksi dan negara dengan jumlah transaksi terbanyak. Tahap EDA membantu dalam memahami karakteristik dataset sebelum dilakukan proses pembersihan data pada tahap berikutnya.

```
print("\n[TAHAP 4.1.2] EXPLORATORY DATA ANALYSIS (EDA)")
print("-">80)

print("\nMENAMPILKAN SAMPLE DATA (5 baris pertama):")
df.show(5)

print("\nSTRUKTUR DATA (Schema):")
df.printSchema()

print("\nSTATISTIK QUANTITY DAN UNITPRICE:")
df.select(
    min("Quantity").alias("MinQuantity"),
    max("Quantity").alias("MaxQuantity"),
    min("UnitPrice").alias("MinUnitPrice"),
    max("UnitPrice").alias("MaxUnitPrice"),
    avg("Quantity").alias("AvgQuantity"),
    avg("UnitPrice").alias("AvgUnitPrice")
).show()

print("\nDISTRIBUSI TRANSAKSI BERDASARKAN NEGARA (Top 10):")

df.groupBy("Country") \
    .count() \
    .orderBy(col("count").desc()) \
    .show(10)
```

Ini output langkah awal proses *Exploratory Data Analysis* (EDA) berupa pemeriksaan 5 baris pertama sampel data mentah. Tahap ini bertujuan untuk memahami struktur dasar, format pengisian, dan kelengkapan kolom data penjualan sebelum dianalisis lebih lanjut. Data yang ditampilkan mencakup informasi utama e-commerce seperti nomor faktur, kode dan deskripsi barang, jumlah beli (*Quantity*), waktu transaksi, harga satuan (*UnitPrice*), ID pelanggan, hingga negara asal pembeli.

```
[TAHAP 4.1.2] EXPLORATORY DATA ANALYSIS (EDA)
-----

MENAMPILKAN SAMPLE DATA (5 baris pertama):
+-----+-----+-----+-----+-----+-----+-----+-----+
| InvoiceNo|StockCode|Description|Quantity|InvoiceDate|UnitPrice|CustomerID|Country|
+-----+-----+-----+-----+-----+-----+-----+-----+
| 536365| 85123A|WHITE HANGING HEA...| 6| 12/01/2010| 2.55| 17850|United Kingdom|
| 536365| 71053| WHITE METAL LANTERN| 6| 12/01/2010| 3.39| 17850|United Kingdom|
| 536365| 84406B|CREAM CUPID HEART...| 8| 12/01/2010| 2.75| 17850|United Kingdom|
| 536365| 84029G|KNITTED UNION FLA...| 6| 12/01/2010| 3.39| 17850|United Kingdom|
| 536365| 84029E|RED WOOLLY HOTTIE...| 6| 12/01/2010| 3.39| 17850|United Kingdom|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Output struktur data atau skema (*Schema*) dari suatu tabel database. Visualisasi ini memperlihatkan daftar nama kolom beserta tipe datanya masing-masing seperti teks (*string*), angka bulat (*integer*), dan angka desimal (*double*). Keterangan nullable = true pada setiap kolom menunjukkan bahwa data tersebut bersifat opsional, artinya kolom-kolom tersebut diperbolehkan kosong atau tidak berisi nilai (*null*). Pemeriksaan skema ini sangat penting di awal analisis untuk memastikan format setiap variabel sudah sesuai sebelum diproses lebih lanjut.

```
STRUKTUR DATA (Schema):
root
|-- InvoiceNo: string (nullable = true)
|-- StockCode: string (nullable = true)
|-- Description: string (nullable = true)
|-- Quantity: integer (nullable = true)
|-- InvoiceDate: string (nullable = true)
|-- UnitPrice: double (nullable = true)
|-- CustomerID: integer (nullable = true)
|-- Country: string (nullable = true)
```

Output ringkasan statistik deskriptif untuk dua variabel utama, yaitu jumlah barang (*Quantity*) dan harga satuan (*UnitPrice*). Ringkasan ini menyajikan nilai minimum, maksimum, serta rata-rata (*average*) dari kedua data tersebut untuk melihat sebaran nilai transaksi awal. Statistik ini langsung mendeteksi adanya keanehan data berupa nilai minus yang sangat ekstrem pada kolom MinQuantity (-80.995) dan MinUnitPrice (-11.062,06). Penemuan nilai negatif ini menjadi indikasi awal yang sangat penting bahwa *dataset* tersebut masih kotor, yang kemungkinan disebabkan oleh adanya transaksi pembatalan (*refund*), retur barang, atau kesalahan input data (*error*), sehingga diperlukan proses pembersihan data (*data cleaning*) sebelum analisis dapat dilanjutkan.

```
STATISTIK QUANTITY DAN UNITPRICE:
+-----+-----+-----+-----+-----+-----+
|MinQuantity|MaxQuantity|MinUnitPrice|MaxUnitPrice|   AvgQuantity|   AvgUnitPrice|
+-----+-----+-----+-----+-----+-----+
|   -80995|    80995|  -11062.06|   38970.0|9.55224954743324|4.611113626089215|
+-----+-----+-----+-----+-----+-----+
```

Output pada gambar ini menampilkan tabel "Distribusi Transaksi Berdasarkan Negara (Top 10)" untuk melihat sebaran geografis aktivitas pembelian pelanggan. Data ini memperlihatkan 10 negara dengan volume transaksi tertinggi, di mana United Kingdom mendominasi pasar secara mutlak dengan total 495.478 transaksi. Sementara itu, posisi berikutnya diikuti oleh negara-negara Eropa lainnya seperti Jerman, Prancis, dan Irlandia (EIRE) dengan angka di kisaran 8.000 hingga 9.000 transaksi. Informasi ini memberikan *insight* penting bagi perusahaan bahwa basis pelanggan terbesar mereka berpusat di Inggris, sehingga strategi pemasaran, distribusi logistik, maupun pengelolaan server dapat difokuskan untuk mendukung wilayah utama tersebut.

DISTRIBUSI TRANSAKSI BERDASARKAN NEGARA (Top 10):

```
+-----+-----+
| Country | count |
+-----+-----+
| United Kingdom | 495478 |
| Germany | 9495 |
| France | 8557 |
| EIRE | 8196 |
| Spain | 2533 |
| Netherlands | 2371 |
| Belgium | 2069 |
| Switzerland | 2002 |
| Portugal | 1519 |
| Australia | 1259 |
+-----+-----+
only showing top 10 rows
```

4.1.3 Penanganan Missing Values

Setelah proses pemeriksaan kualitas data selesai dilakukan, sistem mulai menangani data yang tidak lengkap atau memiliki nilai kosong. Baris transaksi yang tidak memiliki identitas pelanggan (CustomerID) dihapus secara otomatis karena data tersebut tidak dapat digunakan untuk menganalisis perilaku pelanggan maupun pola transaksi.

```
print("\n[TAHAP 4.1.3] PENANGANAN MISSING VALUES")
print("-"*80)

# Hitung missing values sebelum
print("Missing values SEBELUM cleaning:")
for column in df.columns:
    missing_count = df.filter(col(column).isNull()).count()
    if missing_count > 0:
        print(f" - {column}: {missing_count}")

df = df.dropna()
after_dropna = df.count()
print(f"\n✓ Missing values berhasil dihapus")
print(f"✓ Jumlah data setelah dropna: {after_dropna} baris")
print(f"✓ Data yang dihapus: {initial_count - after_dropna} baris")
```

Output proses pembersihan data pada tahap penanganan data yang hilang (*Missing Values*). Sebelum dibersihkan, terdeteksi adanya data kosong pada kolom Description (1.454 data) dan CustomerID (135.080 data). Untuk menjaga kualitas analisis, sistem melakukan tindakan pembersihan dengan menghapus baris data yang kosong tersebut (*dropna*). Hasil akhirnya menunjukkan sebanyak 135.080 baris data kosong berhasil dibuang, sehingga menyisakan total 406.829 baris data yang kini sudah bersih dan siap digunakan untuk proses pemodelan atau analisis berikutnya.

```
[TAHAP 4.1.3] PENANGANAN MISSING VALUES
-----
Missing values SEBELUM cleaning:
  - Description: 1454
  - CustomerID: 135080

✓ Missing values berhasil dihapus
✓ Jumlah data setelah dropna: 406829 baris
✓ Data yang dihapus: 135080 baris
```

4.1.4 Penghapusan Duplikat dan Data Tidak Valid

Setelah penanganan missing values selesai dilakukan, sistem melakukan proses penyaringan data untuk menghapus transaksi yang tidak valid. Penyaringan pertama dilakukan pada kolom Quantity dengan hanya mempertahankan data yang memiliki jumlah barang lebih besar dari nol. Langkah ini bertujuan untuk menghilangkan transaksi retur atau transaksi yang tidak valid. Selanjutnya, sistem melakukan penyaringan pada kolom UnitPrice untuk memastikan seluruh transaksi memiliki harga produk yang valid dan lebih besar dari nol. Tahapan ini penting untuk menjaga konsistensi dan kualitas data sebelum digunakan pada proses analisis lanjutan.

```
print("\n[TAHAP 4.1.4] PENGHAPUSAN DATA TIDAK VALID")
print("-"*80)

print("✓ Memfilter Quantity > 0 (menghapus retur/data tidak valid)...")
before_quantity = df.count()
df = df.filter(col("Quantity") > 0)
after_quantity = df.count()
print(f"  Data yang dihapus: {before_quantity - after_quantity} baris")

print("✓ Memfilter UnitPrice > 0 (memastikan harga valid)...")
before_price = df.count()
df = df.filter(col("UnitPrice") > 0)
after_price = df.count()
print(f"  Data yang dihapus: {before_price - after_price} baris")

print(f"\n✓ Jumlah data setelah filtering: {df.count()} baris")
```

Output proses pembersihan data lanjutan pada tahap "Penghapusan Data Tidak Valid". Proses ini berfokus pada pembersihan nilai negatif ekstrem yang sebelumnya sempat terdeteksi di awal analisis. Sistem melakukan pemfilteran dengan dua aturan utama: pertama, mempertahankan data dengan jumlah barang di atas nol ($\text{Quantity} > 0$) guna menghapus 8.905 baris data retur atau pembatalan transaksi; kedua, memastikan harga satuan di atas nol ($\text{UnitPrice} > 0$) yang berhasil membuang 40 baris data harga yang tidak valid. Setelah seluruh rangkaian penyaringan ini selesai, jumlah data akhir yang tersisa adalah 397.884 baris, yang kini sepenuhnya valid, bersih, dan siap untuk digunakan dalam proses analisis atau pemodelan berikutnya.

```
[TAHAP 4.1.4] PENGHAPUSAN DATA TIDAK VALID
-----
✓ Memfilter Quantity > 0 (menghapus retur/data tidak valid)...
  Data yang dihapus: 8905 baris
✓ Memfilter UnitPrice > 0 (memastikan harga valid)...
  Data yang dihapus: 40 baris

✓ Jumlah data setelah filtering: 397884 baris
```

4.1.5 Laporan Perbandingan Sebelum dan Sesudah Cleaning

Sebagai bentuk evaluasi terhadap proses pembersihan data, sistem menghasilkan laporan perbandingan antara kondisi dataset sebelum dan sesudah dilakukan data cleaning. Laporan ini menampilkan jumlah data awal, jumlah data yang berhasil dibersihkan, serta jumlah data yang dihapus akibat duplikasi, nilai kosong, maupun data tidak valid. Sistem juga menyajikan

statistik ringkas dari beberapa variabel penting untuk menunjukkan peningkatan kualitas data setelah proses pembersihan dilakukan. Tahap ini membantu memastikan bahwa proses cleaning berjalan dengan baik dan memberikan dampak positif terhadap kualitas dataset.

RINGKASAN PROSES DATA CLEANING		
Data Awal	:	541989
Data Setelah Cleaning	:	397884
Total Data Dihapus	:	144825
Persentase Data Tersisa	:	73.4%

4.1.6 Export Dataset ke Apache Parquet

Pada tahap akhir, dataset yang telah bersih dan dilengkapi dengan berbagai fitur tambahan diekspor ke dalam format Apache Parquet. Format Parquet dipilih karena memiliki kemampuan kompresi data yang sangat baik dan mendukung proses pembacaan data secara cepat. Dengan ukuran file yang lebih ringan, proses analisis lanjutan maupun pemodelan machine learning dapat dilakukan dengan lebih efisien.

```
✓ Menyimpan dataset ke format Apache Parquet...
26/05/29 09:22:44 WARN MemoryManager: Total allocation exceeds 95.00% (1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers
26/05/29 09:22:44 WARN MemoryManager: Total allocation exceeds 95.00% (1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/05/29 09:22:44 WARN MemoryManager: Total allocation exceeds 95.00% (1,020,054,720 bytes) of heap memory
Scaling row group sizes to 76.00% for 10 writers
26/05/29 09:22:44 WARN MemoryManager: Total allocation exceeds 95.00% (1,020,054,720 bytes) of heap memory
Scaling row group sizes to 69.00% for 11 writers
26/05/29 09:22:46 WARN MemoryManager: Total allocation exceeds 95.00% (1,020,054,720 bytes) of heap memory
Scaling row group sizes to 76.00% for 10 writers
26/05/29 09:22:46 WARN MemoryManager: Total allocation exceeds 95.00% (1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/05/29 09:22:46 WARN MemoryManager: Total allocation exceeds 95.00% (1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers
✓ Dataset berhasil disimpan ke: data/cleaned_data
✓ Jumlah baris akhir: 397884
✓ Jumlah kolom: 9
```

4.2 Feature Engineering dengan PySpark

Setelah proses data cleaning selesai dilakukan, tahap berikutnya adalah feature engineering. Tahap ini bertujuan untuk menambahkan fitur-fitur baru yang dapat memperkaya informasi pada dataset sehingga proses analisis data dan machine learning dapat dilakukan dengan lebih baik.

Pada penelitian ini, feature engineering dilakukan menggunakan Apache Spark dan library PySpark yang diimplementasikan pada file `feature_engineering.py`. Dataset hasil cleaning sebelumnya dibaca kembali dari folder `data/cleaned_data` untuk kemudian dilakukan proses transformasi dan penambahan fitur baru.

4.2.1 Perhitungan Nilai Transaksi

Pada tahap ini sistem menambahkan kolom baru bernama `TotalPrice`. Kolom tersebut dihitung menggunakan perkalian antara jumlah barang pada kolom `Quantity` dan harga satuan produk pada kolom `UnitPrice`. Variabel `TotalPrice` digunakan untuk merepresentasikan total nilai transaksi pada setiap pembelian pelanggan dan menjadi salah satu fitur utama pada proses analisis transaksi serta deteksi anomali.

```
df = df.withColumn(
    "TotalPrice",
    col("Quantity") * col("UnitPrice")
)
```

Output proses rekayasa fitur pada tahap "Perhitungan Nilai Transaksi". Pada tahap ini, sebuah kolom baru bernama TotalPrice berhasil dibuat secara otomatis dengan menggunakan formula perkalian antara jumlah barang dan harga satuan, yaitu:

$$\text{TotalPrice} = \text{Quantity} \times \text{UnitPrice}$$

Sistem menyajikan sampel hasil perhitungan untuk 10 baris pertama beserta ringkasan statistik dari fitur baru tersebut. Berdasarkan data statistik TotalPrice yang dihasilkan, nilai transaksi paling kecil (*Min*) tercatat sebesar 0.001, nilai transaksi terbesar (*Max*) mencapai 168,469.6, dengan rata-rata pengeluaran pelanggan (*Average*) sebesar 22.39, serta standar deviasi (*StdDev*) di angka 309.07. Pembuatan kolom total harga ini sangat krusial karena menjadi variabel kunci yang membantu model Machine Learning dalam mengenali nominal transaksi yang tidak wajar atau bernilai ekstrem.

```
✓ Kolom TotalPrice berhasil dibuat
Formula: TotalPrice = Quantity * UnitPrice

Sample TotalPrice (10 baris):
+-----+-----+-----+
|Quantity|UnitPrice|TotalPrice|
+-----+-----+-----+
|6|2.55|15.299999999999999|
|6|3.39|20.34|
|8|2.75|22.0|
|6|3.39|20.34|
|6|3.39|20.34|
|2|7.65|15.3|
|6|4.25|25.5|
|6|1.85|11.100000000000001|
|6|1.85|11.100000000000001|
|32|1.69|54.08|
+-----+-----+-----+

Statistik TotalPrice:
+-----+-----+-----+-----+
|Min|Max|Average|StdDev|
+-----+-----+-----+-----+
|0.001|168469.6|22.396999889415056|309.0710414140188|
+-----+-----+-----+-----+
```

4.2.2 Pembacaan Dataset Hasil Cleaning

Pada tahap feature engineering, sistem membentuk beberapa fitur baru yang berasal dari data transaksi pelanggan. Fitur-fitur tersebut dibuat untuk membantu sistem dalam memahami pola transaksi, perilaku pelanggan, serta karakteristik produk yang dijual.

```

print("\n[TAHAP 4.2.1] MEMBACA DATA CLEANED")
print("-"*88)

df = spark.read.parquet(["data/cleaned_data"])

print(f"✓ Data berhasil dibaca dari: data/cleaned_data")
print(f"✓ Jumlah baris: {df.count()}")
print(f"✓ Jumlah kolom: {len(df.columns)}")
print(f"\nKolom yang tersedia:")
for col_name in df.columns:
    print(f"  - {col_name}")
print()

```

Output awal dari tahapan baru, yaitu "Membaca Data Cleaned". Berfungsi untuk memastikan bahwa data yang telah melalui seluruh rangkaian proses pembersihan sebelumnya telah berhasil disimpan dan dimuat kembali dengan benar. Sistem mencatat data berhasil dibaca dari direktori data/cleaned_data dengan kondisi yang sudah rapi, yaitu memiliki total 397.884 baris dan kini berkembang menjadi 9 kolom. Sembilan kolom yang siap digunakan ini meliputi variabel asli seperti *InvoiceNo*, *StockCode*, *Description*, *Quantity*, *InvoiceDate*, *UnitPrice*, *CustomerID*, dan *Country*, serta satu kolom tambahan hasil rekayasa fitur sebelumnya, yaitu *TotalPrice*.

```

✓ Data berhasil dibaca dari: data/cleaned_data
✓ Jumlah baris: 397884
✓ Jumlah kolom: 9

Kolom yang tersedia:
- InvoiceNo
- StockCode
- Description
- Quantity
- InvoiceDate
- UnitPrice
- CustomerID
- Country
- TotalPrice

```

4.2.3 Pembuatan Fitur Waktu Transaksi

Pada tahap ini, sistem melakukan konversi kolom *InvoiceDate* dari format string menjadi format timestamp menggunakan fungsi `to_timestamp()`. Proses ini dilakukan agar data waktu dapat diproses secara kronologis. Setelah berhasil dikonversi, sistem membentuk beberapa fitur waktu baru seperti *Hour*, *Day*, *MonthNum*, *WeekdayNum*, dan *Month*. Fitur waktu digunakan untuk membantu sistem dalam menganalisis pola transaksi pelanggan berdasarkan waktu pembelian, seperti jam transaksi paling aktif maupun tren transaksi bulanan.

```

print("\n[TAHAP 4.2.2] PEMBUATAN FITUR WAKTU TRANSAKSI")
print("-"*88)

# Parse InvoiceDate dengan format "M/dd/yyyy H:mm"
df = df.withColumn("InvoiceDate", to_timestamp(col("InvoiceDate"), "M/dd/yyyy H:mm"))
print("✓ InvoiceDate berhasil dikonversi ke format timestamp")

df = df.withColumn("Hour", hour(col("InvoiceDate")).cast("double"))
df = df.withColumn("Day", dayofmonth(col("InvoiceDate")).cast("double"))
df = df.withColumn("MonthNum", month(col("InvoiceDate")).cast("double"))
df = df.withColumn("WeekdayNum", (dayofweek(col("InvoiceDate")) - 1).cast("double"))
df = df.withColumn("Month", date_format(col("InvoiceDate"), "yyyy-MM"))

print("✓ Fitur waktu berhasil dibuat: Hour, Day, MonthNum, WeekdayNum, Month")
print("\nSample data fitur waktu (5 baris pertama):")
df.select("InvoiceDate", "Hour", "Day", "MonthNum", "WeekdayNum", "Month").show(5, truncate=False)

print("\nStatistik fitur waktu:")
df.select(
    min("Hour").alias("Min_Hour"),
    max("Hour").alias("Max_Hour"),
    min("Day").alias("Min_Day"),
    max("Day").alias("Max_Day"),
    min("MonthNum").alias("Min_Month"),
    max("MonthNum").alias("Max_Month")
).show()
print()

```

Output dari kelanjutan tahap rekayasa fitur pada bagian "Pembuatan Fitur Waktu Transaksi". Proses ini diawali dengan mengonversi kolom teks InvoiceDate ke dalam format *timestamp* agar terbaca sebagai data penanggalan yang valid oleh sistem.

Dari hasil konversi tersebut, sistem berhasil memecah dan membuat lima kolom fitur waktu baru secara spesifik, yaitu jam (*Hour*), tanggal (*Day*), nomor bulan (*MonthNum*), nomor hari dalam seminggu (*WeekdayNum*), serta kombinasi tahun-bulan (*Month*). Di bagian bawah, statistik fitur menunjukkan rentang waktu operasional transaksi, di mana jam transaksi aktif berada di antara pukul 06.00 hingga 20.00, rentang tanggal dari tanggal 13 sampai 31, dan mencakup bulan ke-1 hingga ke-12. Ekstraksi komponen waktu ini sangat krusial untuk menganalisis pola perilaku musiman pelanggan serta membantu model mendeteksi waktu-waktu rawan terjadinya transaksi anomali.

```

[TAHAP 4.2.2] PEMBUATAN FITUR WAKTU TRANSAKSI
-----
✓ InvoiceDate berhasil dikonversi ke format timestamp
✓ Fitur waktu berhasil dibuat: Hour, Day, MonthNum, WeekdayNum, Month

Sample data fitur waktu (5 baris pertama):
+-----+-----+-----+-----+-----+
|InvoiceDate|Hour|Day|MonthNum|WeekdayNum|Month|
+-----+-----+-----+-----+-----+
|2011-09-25 14:08:00|14.0|25.0|9.0|0.0|2011-09|
|2011-09-25 14:08:00|14.0|25.0|9.0|0.0|2011-09|
|2011-09-25 14:08:00|14.0|25.0|9.0|0.0|2011-09|
|2011-09-25 14:08:00|14.0|25.0|9.0|0.0|2011-09|
|2011-09-25 14:08:00|14.0|25.0|9.0|0.0|2011-09|
+-----+-----+-----+-----+-----+
only showing top 5 rows

Statistik fitur waktu:
+-----+-----+-----+-----+-----+
|Min_Hour|Max_Hour|Min_Day|Max_Day|Min_Month|Max_Month|
+-----+-----+-----+-----+-----+
|6.0|20.0|13.0|31.0|1.0|12.0|
+-----+-----+-----+-----+-----+

```

4.2.4 Pembuatan Revenue Category

Sistem kemudian membentuk fitur RevenueCategory berdasarkan nilai transaksi pada kolom TotalPrice. Fitur ini digunakan untuk mengelompokkan tingkat nilai transaksi pelanggan ke dalam beberapa kategori.

```
# -----  
# REVENUE CATEGORY  
# -----  
df = df.withColumn(  
    "RevenueCategory",  
    when(col("TotalPrice") > 1000, "High")  
    .when(col("TotalPrice") > 300, "Medium")  
    .otherwise("Low")  
)
```

Kategori transaksi dibagi menjadi:

- High untuk transaksi dengan TotalPrice lebih dari 1000
- Medium untuk transaksi dengan TotalPrice lebih dari 300
- Low untuk transaksi dengan TotalPrice kurang dari atau sama dengan 300

Fitur ini membantu sistem dalam membedakan transaksi bernilai tinggi dan transaksi normal pada proses analisis.

Distribusi RevenueCategory:

```
+-----+  
|RevenueCategory| count|  
+-----+  
|          Low|395803|  
|         Medium| 1766|  
|          High|   315|  
+-----+
```

4.2.5 Encoding Data Negara

Kolom Country yang masih berbentuk data kategorikal diubah menjadi bentuk numerik menggunakan metode StringIndexer dari Spark MLlib. Hasil proses encoding disimpan pada kolom baru bernama CountryEncoded. Transformasi ini bertujuan agar data negara dapat digunakan pada proses machine learning dan analisis statistik yang membutuhkan data numerik.

```
# -----  
# COUNTRY ENCODING  
# -----  
if "Country" in df.columns:  
    indexer = StringIndexer(  
        inputCol="Country",  
        outputCol="CountryEncoded",  
        handleInvalid="keep"  
    )  
    df = indexer.fit(df).transform(df)
```

Ini output proses *encoding* data kategorikal pada tahap persiapan data, di mana kolom asal negara (Country) berhasil diubah menjadi format numerik baru bernama CountryEncoded. Tabel tersebut menyajikan sampel pemetaan (*mapping*) untuk 15 negara, contohnya United Kingdom diubah menjadi angka 0.0, Germany menjadi 1.0, France menjadi 2.0, hingga Cyprus menjadi 14.0. Proses *encoding* ini sangat krusial dalam tahap *data preprocessing* karena model Machine Learning hanya dapat membaca dan mengolah data yang berbentuk angka (numerik), bukan teks atau kategori mentah.

Country berhasil di-encode menjadi CountryEncoded (numerik)

Mapping Country ke CountryEncoded (Sample 15 negara):

Country	CountryEncoded
United Kingdom	0.0
Germany	1.0
France	2.0
IRE	3.0
Spain	4.0
Netherlands	5.0
Belgium	6.0
Switzerland	7.0
Portugal	8.0
Australia	9.0
Norway	10.0
Italy	11.0
Channel Islands	12.0
Finland	13.0
Cyprus	14.0

4.2.6 Pembuatan Product Features

Pada tahap ini, sistem membentuk fitur tambahan yang berkaitan dengan produk menggunakan kolom StockCode. Sistem menghitung jumlah transaksi pada setiap produk menggunakan fungsi `partitionBy()` dan menghasilkan fitur `ProductFrequency`. Fitur ini menunjukkan seberapa sering suatu produk muncul dalam transaksi pelanggan. Informasi tersebut membantu sistem dalam mengenali produk dengan tingkat penjualan tinggi maupun produk yang jarang dibeli pelanggan.

```
#  
# PRODUCT FEATURES  
#  
if "StockCode" in df.columns:  
    stock_window = Window.partitionBy("StockCode")  
    df = df.withColumn(  
        "ProductFrequency",  
        count("StockCode").over(stock_window)  
    )
```

Output dari kelanjutan proses rekayasa fitur pada tahapan "*Pembuatan Product Features*". Pada tahap ini, sebuah kolom fitur baru bernama `ProductFrequency` berhasil dibuat untuk menghitung akumulasi atau jumlah transaksi per produk berdasarkan kode barang (`StockCode`). Berdasarkan ringkasan statistik yang disajikan, frekuensi penjualan produk memiliki nilai minimum 1 kali transaksi, nilai maksimum mencapai 2.035 kali transaksi, dengan rata-rata (*Avg_Frequency*) berada di angka 355,51. Di bawahnya, terdapat tabel "Top

10 produk paling sering dijual" yang memperlihatkan kode barang dengan likuiditas tertinggi, di mana produk dengan kode 85123A menduduki posisi pertama dengan total 2.035 kali transaksi. Informasi mengenai tingkat popularitas atau frekuensi kemunculan produk ini sangat membantu model dalam mengenali jika ada aktivitas pembelian tidak wajar terhadap produk-produk tertentu.

```
[TAHAP 4.2.5] PEMBUATAN PRODUCT FEATURES
-----
✓ ProductFrequency berhasil dibuat
Menghitung: Jumlah transaksi per produk (StockCode)

Statistik ProductFrequency:
-----+-----+-----+
|Min_Frequency|Max_Frequency| Avg_Frequency|
-----+-----+-----+
|          1|         2035|355.5176282534608|
-----+-----+-----+

Top 10 produk paling sering dijual:
-----+-----+
|StockCode|Frequency|
-----+-----+
| 85123A|    2035|
| 22423|    1723|
| 85099B|    1618|
| 84879|    1408|
| 47566|    1396|
| 20725|    1317|
| 22720|    1159|
| 20727|    1105|
| POST|    1099|
| 23203|    1098|
-----+-----+
```

4.2.7 Pembuatan Customer Features

Selain fitur produk, sistem juga membentuk beberapa fitur yang berkaitan dengan perilaku pelanggan menggunakan kolom CustomerID.

Fitur yang dihasilkan meliputi:

- AvgCustomerSpend untuk menghitung rata rata pengeluaran pelanggan
- CustomerFrequency untuk menghitung jumlah transaksi pelanggan
- CustomerTotalQty untuk menghitung total barang yang dibeli pelanggan
- CustomerPriceStd untuk menghitung standar deviasi nilai transaksi pelanggan

Fitur fitur tersebut membantu sistem dalam memahami karakteristik dan perilaku belanja pelanggan secara lebih mendalam.

```

cust_window = Window.partitionBy("CustomerID")

df = df.withColumn(
    "AvgCustomerSpend",
    avg("TotalPrice").over(cust_window)
)

df = df.withColumn(
    "CustomerFrequency",
    count("CustomerID").over(cust_window)
)

df = df.withColumn(
    "CustomerTotalQty",
    sum("Quantity").over(cust_window)
)

df = df.withColumn(
    "CustomerPriceStd",
    stddev("TotalPrice").over(cust_window)
)

```

Output pada tahap "Pembuatan *Customer Features*" untuk memperkaya data sebelum masuk ke model Machine Learning. Sistem berhasil membuat empat fitur baru berbasis perilaku pelanggan, yaitu AvgCustomerSpend (rata-rata pengeluaran), CustomerFrequency (total transaksi), CustomerTotalQty (total barang yang dibeli), dan CustomerPriceStd (variasi nilai transaksi). Pembuatan fitur ini sangat penting untuk membangun profil kebiasaan belanja pengguna, sehingga sistem dapat dengan mudah mendeteksi jika muncul transaksi baru yang polanya menyimpang jauh dari batas normal.

[TAHAP 4.2.6] PEMBUATAN CUSTOMER FEATURES

✓ Customer Features berhasil dibuat:

- AvgCustomerSpend: Rata-rata pengeluaran per pelanggan
- CustomerFrequency: Jumlah transaksi per pelanggan
- CustomerTotalQty: Total barang yang dibeli pelanggan
- CustomerPriceStd: Standar deviasi nilai transaksi pelanggan

Output pada tahap "Statistik *Customer Features*" berupa ringkasan deskriptif (min, max, rata-rata, standar deviasi) untuk mengevaluasi sebaran data dari empat fitur baru pelanggan. Ringkasan statistik ini penting untuk memberikan gambaran batas normal perilaku belanja pelanggan, sekaligus mempermudah model Machine Learning dalam mengenali pencilan (*outliers*) yang berpotensi sebagai transaksi anomali.

Sample Customer Features (10 pelanggan unik):

CustomerID	AvgCustomerSpend	CustomerFrequency	CustomerTotalQty	CustomerPriceStd
12347	23.681318681318672	182	2458	23.289982318581162
12349	24.876827397268273	73	631	34.655912784589584
12355	35.33846153846154	13	248	29.78352219466962
12356	47.65135593228339	59	1591	48.5722968956169
12358	61.47684218526315	19	248	52.27752321984825
12368	28.636124831887746	129	1165	21.384617726788553
12362	19.64748128388752	266	2229	9.66467128285669
12365	29.153636363636362	22	174	65.1966351414414
12367	15.354545454545455	11	173	5.648958869655613
12373	26.842857142857144	14	197	18.268282968486132

4.2.8 Penyimpanan Dataset Hasil Feature Engineering

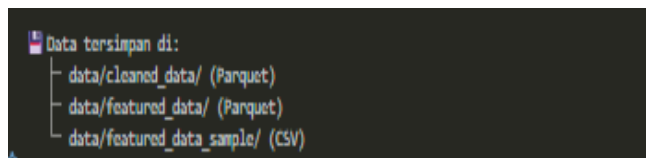
Pada tahap akhir, dataset hasil feature engineering disimpan kembali ke dalam format Apache Parquet pada folder data/featured_data. Dataset ini menjadi sumber data utama yang digunakan pada proses deteksi anomali, customer segmentation, customer behavior analytics, serta visualisasi dashboard pada tahapan berikutnya.

```
# -----  
# SAVE TO PARQUET  
# -----  
print("MENYIMPAN FEATURED DATA (PARQUET)")  
  
df.write.mode("overwrite").parquet("data/featured_data")  
  
print("FEATURE ENGINEERING SELESAI")  
  
spark.stop()
```

Output menampilkan lokasi dan format penyimpanan data setelah melalui proses pembersihan dan rekayasa fitur. Data berhasil disimpan ke dalam tiga direktori terpisah sesuai kebutuhan:

- data/cleaned_data/ (Parquet): Menyimpan data bersih dalam format Parquet untuk efisiensi penyimpanan.
- data/featured_data/ (Parquet): Menyimpan *dataset* final yang sudah lengkap dengan fitur-fitur baru untuk pemodelan.
- data/featured_data_sample/ (CSV): Menyimpan sampel data dalam format CSV agar mudah dibuka dan ditinjau secara manual.

Penyimpanan ini memastikan semua tahapan pengolahan data terdokumentasi dengan format yang optimal untuk proses pelatihan model Machine Learning selanjutnya.



```
Data tersimpan di:  
├── data/cleaned_data/ (Parquet)  
├── data/featured_data/ (Parquet)  
└── data/featured_data_sample/ (CSV)
```

4.3 Data Transformation

4.3.1 Konversi Tipe Data Numerik

Pada tahap berikutnya, sistem memastikan seluruh fitur numerik memiliki tipe data yang sesuai menggunakan proses casting ke tipe double. Proses ini dilakukan pada beberapa fitur penting seperti Quantity, UnitPrice, TotalPrice, Hour, ProductFrequency, AvgCustomerSpend, dan fitur numerik lainnya agar dapat diproses dengan baik pada proses analisis statistik maupun machine learning.

```
for c in numeric_cols:  
    if c in df.columns:  
        df = df.withColumn(c, col(c).cast("double"))
```

Output tahap "Kombinasi Semua Fitur (Data Final)" sebagai langkah akhir dari *data preprocessing* dan *feature engineering*. Pada tahap ini, sistem menyatukan seluruh variabel asli dan fitur baru menjadi satu *dataset* final yang rapi dengan total 397.884 baris dan 15 kolom.

Daftar 15 kolom final tersebut meliputi:

- Kolom Transaksi Asli: *InvoiceNo*, *StockCode*, *Description*, *Quantity*, *InvoiceDate*, *UnitPrice*, *CustomerID*, dan *Country*.
- Kolom Fitur Rekayasa: *TotalPrice*, *Hour*, *Day*, *MonthNum*, *WeekdayNum*, *CountryEncoded*, dan *ProductFrequency*.

Penyusunan ini menandai berakhirnya tahap manipulasi data, sehingga *dataset* kini sudah dalam format ideal dan siap dimasukkan ke dalam model Machine Learning untuk proses pelatihan (*training*) deteksi transaksi anomali.

```
[TAHAP 4.2.8] KONVERSI TIPE DATA NUMERIK
-----
✓ Mengkonversi tipe data ke Double:
- Quantity: ✓
- UnitPrice: ✓
- TotalPrice: ✓
- Hour: ✓
- Day: ✓
- MonthNum: ✓
- WeekdayNum: ✓
- CountryEncoded: ✓
- ProductFrequency: ✓
- AvgCustomerSpend: ✓
- CustomerFrequency: ✓
- CustomerTotalQty: ✓
- CustomerPriceStd: ✓
```

4.4 Integrasi Database

Setelah seluruh data selesai diolah, dilengkapi dengan fitur baru, dan diberi label anomali oleh model machine learning, tahap selanjutnya adalah menyimpan hasil akhir tersebut. Penyimpanan ini harus dilakukan dengan aman agar data mudah diakses dan ditampilkan ke layar pengguna. Proyek ini tidak hanya bergantung pada satu jenis penyimpanan. Untuk menampung kompleksitas data, sistem menggunakan strategi penyimpanan ganda (hybrid storage), yaitu menggabungkan basis data relasional (PostgreSQL) dan basis data non-relasional (MongoDB).

4.4.1 PostgreSQL - Database Relasional.

PostgreSQL digunakan sebagai database relasional utama untuk menyimpan seluruh data transaksi retail yang telah diproses menggunakan Apache Spark. Dataset yang disimpan berasal dari folder: `data/featured_data`.

Dataset tersebut berisi:

- a. Data transaksi retail
- b. Fitur waktu transaksi
- c. Fitur pelanggan
- d. Fitur produk
- e. Kategori pendapatan transaksi

Penyimpanan dilakukan menggunakan JDBC PostgreSQL yang dihubungkan langsung melalui Apache Spark.

1. Implementasi Penyimpanan Data ke PostgreSQL

Proses dimulai dengan membaca dataset hasil feature engineering menggunakan Apache Spark.

```
data_path = "data/featured_data"

try:
    df = spark.read.parquet(data_path)
```

Kemudian sistem melakukan konfigurasi koneksi PostgreSQL.

```
postgres_url = "jdbc:postgresql://localhost:5432/retail_db"
postgres_properties = {
    "user": "postgres",
    "password": "samuel",
    "driver": "org.postgresql.Driver"
}
```

Data kemudian dikirim ke PostgreSQL menggunakan JDBC Spark.

```
df.write \
    .mode("overwrite") \
    .jdbc(
        url=postgres_url,
        table=table_name,
        properties=postgres_properties
    )
```

2. Output Proses Penyimpanan PostgreSQL

Saat program dijalankan menggunakan: `python spark/save_postgres.py` sistem menghasilkan output sebagai berikut:

```
✓ Data berhasil dibaca dari: data/featured_data
✓ Jumlah baris data: 397,884
✓ Jumlah kolom: 21
```

Output tersebut menunjukkan bahwa Spark berhasil membaca dataset hasil feature engineering.

Sebelum proses penyimpanan dilakukan, sistem melakukan validasi data. Output validasi:

```
Verifikasi Tipe Data Numerik:
✓ Quantity: double
✓ UnitPrice: double
✓ TotalPrice: double
✓ Hour: double
✓ Day: double
✓ MonthNum: double
✓ WeekdayNum: double
✓ CountryEncoded: double
✓ ProductFrequency: double
✓ AvgCustomerSpend: double
✓ CustomerFrequency: double
✓ CustomerTotalQty: double
✓ CustomerPriceStd: double
```

3. Statistik Data PostgreSQL

```
Distribusi berdasarkan Revenue Category:
+-----+
|RevenueCategory| count|
+-----+
|           High|   315|
|           Low|395803|
|          Medium|  1766|
+-----+
```

Output tersebut menunjukkan bahwa sebagian besar transaksi termasuk kategori Low Revenue.

```
Top 5 Negara dengan Transaksi Terbanyak:
+-----+
|      Country| count|
+-----+
|United Kingdom|354321|
|      Germany|   9840|
|      France|   8341|
|        EIRE|   7236|
|       Spain|   2484|
+-----+
```

Output tersebut menunjukkan 5 top negara dengan transaksi terbanyak.

4.4.2 MongoDB - Database Non-Relasional

Sebagai pelengkap basis data relasional, sistem juga mengintegrasikan MongoDB sebagai basis data non-relasional (NoSQL database).

1. Tujuan: MongoDB berfungsi sebagai ruang penyimpanan berbasis dokumen (document-oriented database) yang menampung data transaksi mencurigakan dalam koleksi bernama retail_anomalies.

2. Implementasi Anomali Detection

Model Isolation Forest dijalankan menggunakan tiga fitur utama:

```
# =====
# FEATURES UNTUK ANOMALY DETECTION
# =====

features = pdf[
    [
        "Quantity",
        "UnitPrice",
        "TotalPrice"
    ]
]
```

Ketiga fitur tersebut dipilih karena secara langsung merepresentasikan karakteristik transaksi retail.

Model dijalankan menggunakan:

```
# =====  
# ISOLATION FOREST MODEL  
# =====  
  
model = IsolationForest(  
    contamination=0.02,  
    random_state=42,  
    n_estimators=100  
)
```

Hasil Label Anomali Hasil model menghasilkan: -1 = anomali, 1 = normal. Kemudian sistem mengubah hasil tersebut menjadi:

```
# =====  
# PREDICT ANOMALY  
# =====  
  
predictions = model.predict(features)  
  
pdf["anomaly"] = np.where(  
    predictions == -1,  
    1,  
    0  
)
```

3. Penjelasan Hasil Model

Untuk memberikan interpretasi yang lebih mudah dipahami, sistem menambahkan aturan tambahan berbasis kondisi bisnis:

```
pdf["anomaly_reason"] = "Normal"  
  
pdf.loc[  
    pdf["Quantity"] > 100,  
    "anomaly_reason"  
] = "Large Quantity"  
  
pdf.loc[  
    pdf["TotalPrice"] > 1000,  
    "anomaly_reason"  
] = "High Transaction"  
  
pdf.loc[  
    pdf["UnitPrice"] > 100,  
    "anomaly_reason"  
] = "Expensive Product"
```

Kategori penjelasan:

- Large Quantity → jumlah barang tidak wajar
- High Transaction → nilai transaksi terlalu tinggi
- Expensive Product → harga produk terlalu tinggi

```
JUMLAH DATA SETELAH DROPNA SUBSET:  
397884  
MENJALANKAN ISOLATION FOREST  
TOTAL DATA:  
397884  
TOTAL ANOMALY:  
7767  
ANOMALY PERCENTAGE:  
1.95%
```

4. Keunggulan yang Diharapkan:
 - a. Struktur Data Fleksibel (Flexible Schema): Jika di kemudian hari tim pemeriksa penipuan (fraud analyst) ingin menambahkan catatan investigasi khusus pada satu transaksi aneh tertentu, mereka bisa langsung menambahkan kolom baru tanpa merusak struktur data transaksi lainnya.
 - b. Penyimpanan Data Kompleks: Mendukung penyimpanan data yang bertingkat (nested data), sehingga sangat cocok untuk menampung rincian riwayat transaksi yang rumit.

4.4.3 Cross-System Query - Verifikasi Konsistensi Data

Karena sistem memecah penyimpanannya ke dalam dua tempat yang berbeda, muncul tantangan baru: bagaimana memastikan data di PostgreSQL dan MongoDB selalu selaras? Untuk menjawab hal ini, sistem menerapkan langkah pemeriksaan silang (cross-check).

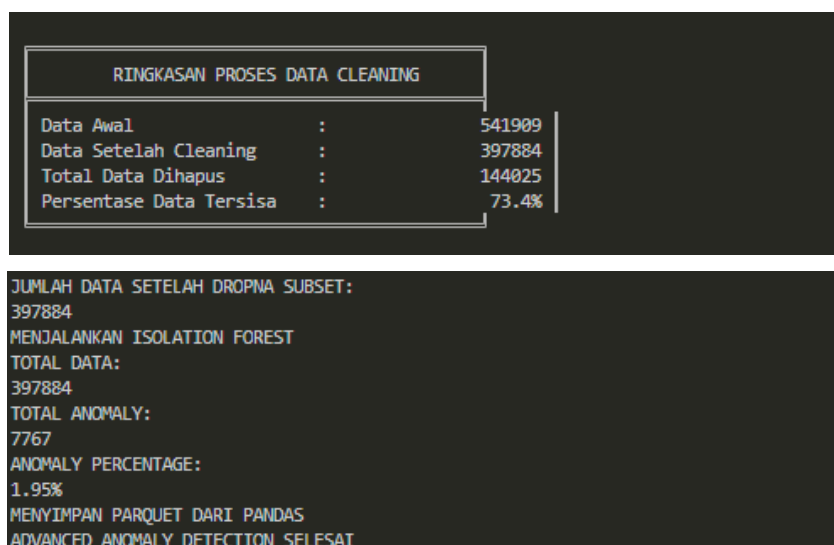
1. Tujuan: Memastikan bahwa tidak ada kegagalan pengiriman data dari sistem Spark menuju basis data.
2. Proses Implementasi: Aplikasi dashboard di program agar selalu menarik dan membaca data dari PostgreSQL dan MongoDB pada waktu yang bersamaan . Data ini kemudian dihubungkan (join) untuk melihat kecocokannya .
3. Indikator Keberhasilan: Langkah ini sangat berguna untuk memantau masalah (troubleshooting). Jika saat halaman dashboard dibuka grafik data anomali dari MongoDB terlihat kosong, hal tersebut langsung menjadi penanda atau peringatan bahwa proses pipeline pengiriman data ke MongoDB belum selesai dijalankan secara penuh, atau terjadi salah pengetikan nama basis datanya (misalnya seharusnya mengakses retail_db) .

BAB V

HASIL DAN ANALISIS

5.1 Hasil Data Cleaning

Setelah sistem dijalankan, proses pembersihan terbukti berhasil menyaring puluhan ribu data kotor. Dari file mentah awal, sistem berhasil menyisakan 397.884 baris data transaksi yang bersih dan valid. Melalui tahap penambahan fitur (feature engineering), data ini diperluas hingga memiliki 21 kolom informasi. Pada tahapan deteksi machine learning, model Isolation Forest mendeteksi adanya 7.767 transaksi anomali dari keseluruhan total 397.884 transaksi tersebut.

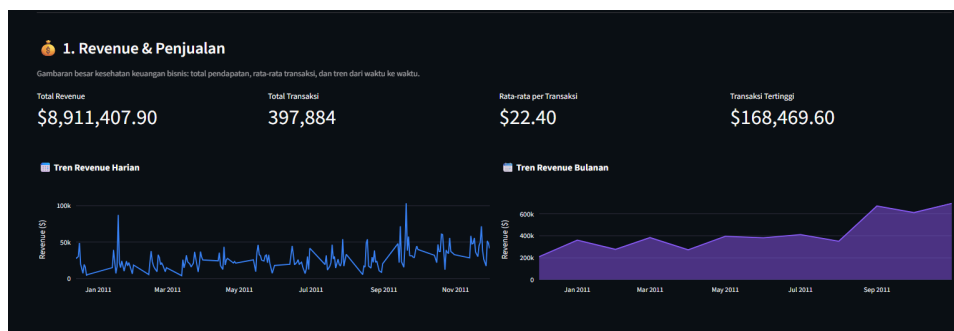


5.2 Insight Bisnis

Data yang telah diproses kemudian diubah menjadi grafik interaktif di dashboard yang memberikan pemahaman bisnis nyata bagi perusahaan, di antaranya:

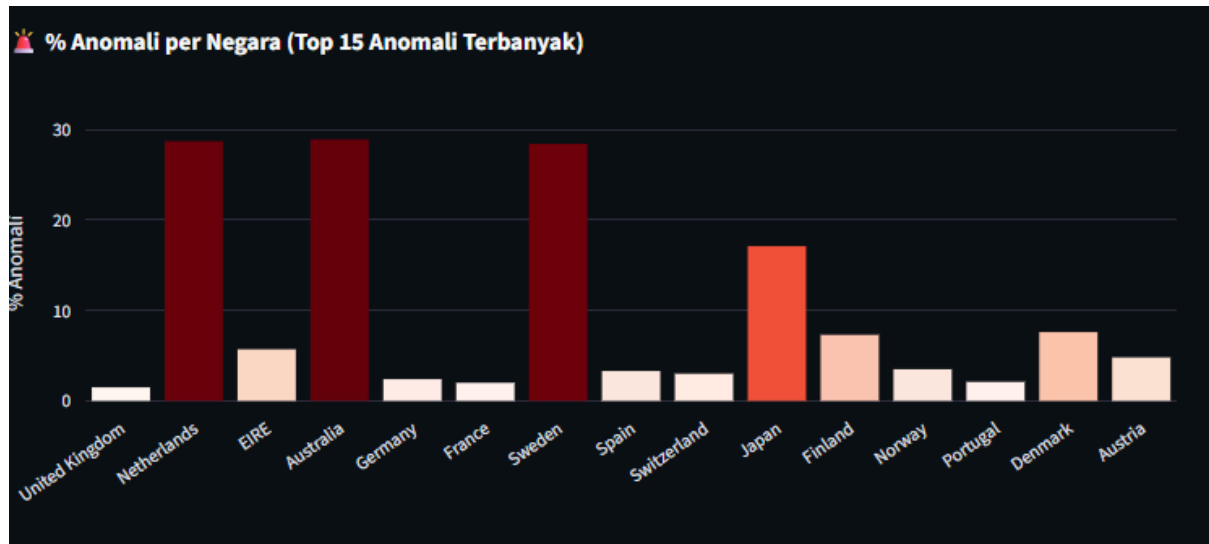
5.2.1 Pendapatan dan Penjualan (Revenue)

Dashboard secara otomatis menghitung total pendapatan bersih perusahaan serta rata-rata nilai transaksi pada setiap pesanan pelanggan. Informasi tersebut disajikan dalam bentuk grafik tren harian dan bulanan untuk mempermudah manajemen dalam memantau perkembangan penjualan, baik peningkatan maupun penurunan yang terjadi dari waktu ke waktu.



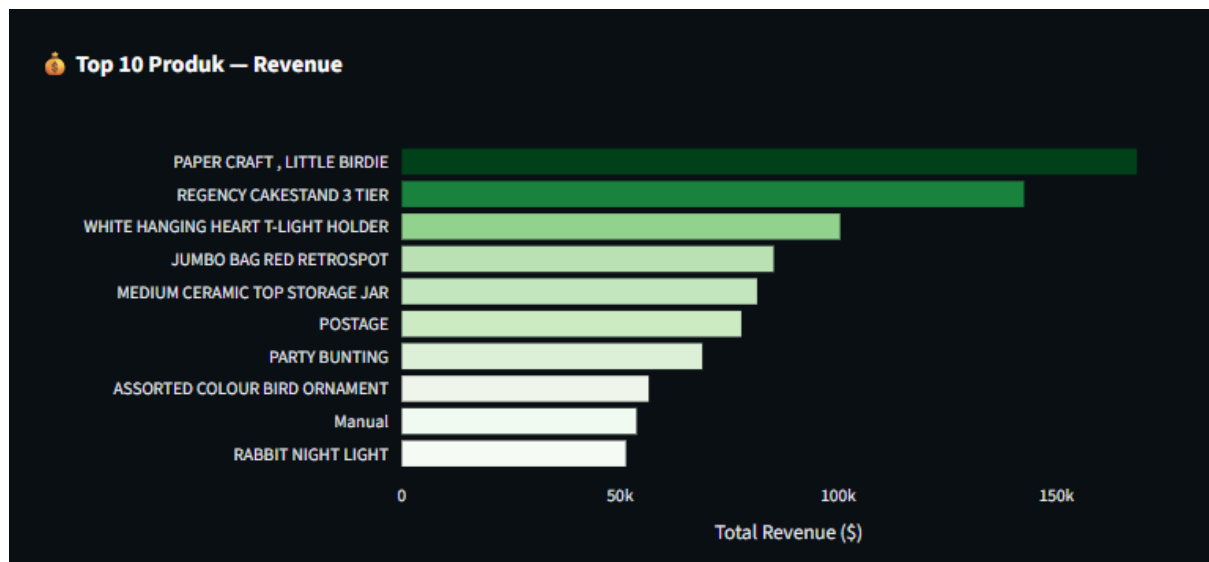
5.2.2 Distribusi Geografis dan Lokasi Risiko

Sistem menampilkan visualisasi asal negara pelanggan dalam bentuk diagram distribusi geografis. Melalui visualisasi ini, perusahaan dapat mengetahui negara dengan jumlah transaksi tertinggi sekaligus wilayah yang memiliki tingkat transaksi anomali paling tinggi. Informasi tersebut membantu tim Fraud Analyst dalam meningkatkan pengawasan terhadap wilayah yang dianggap berisiko.



5.2.3 Analisis Produk Terlaris

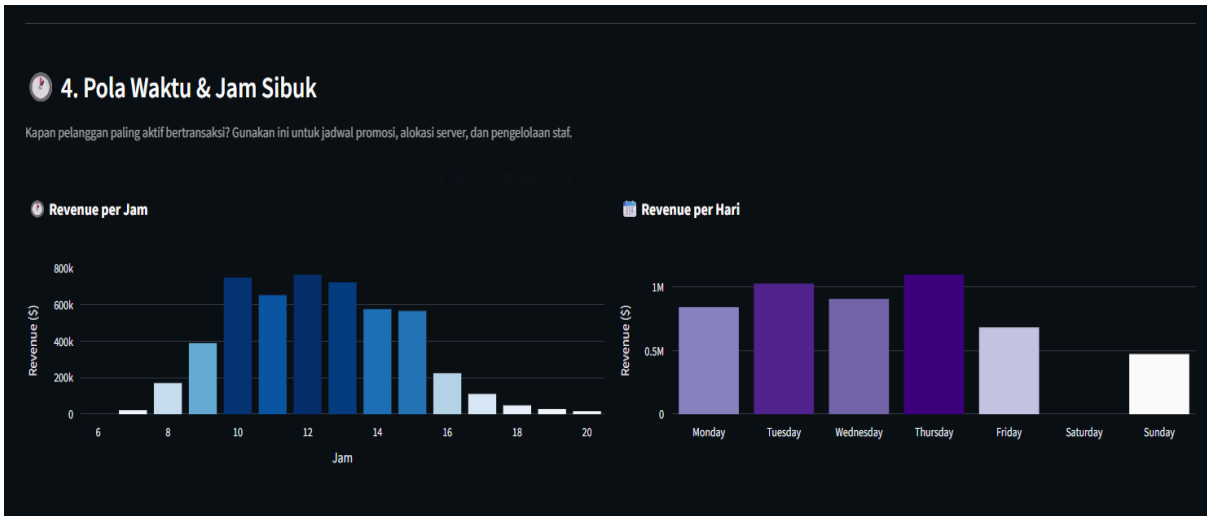
Sistem mampu menganalisis tingkat popularitas setiap produk dan menampilkan daftar 10 produk dengan volume penjualan tertinggi. Informasi ini memberikan wawasan operasional bagi bagian inventaris untuk memprioritaskan penyediaan stok produk unggulan sehingga risiko kehabisan persediaan dapat diminimalkan.



5.2.4 Pola Waktu dan Jam Sibuk

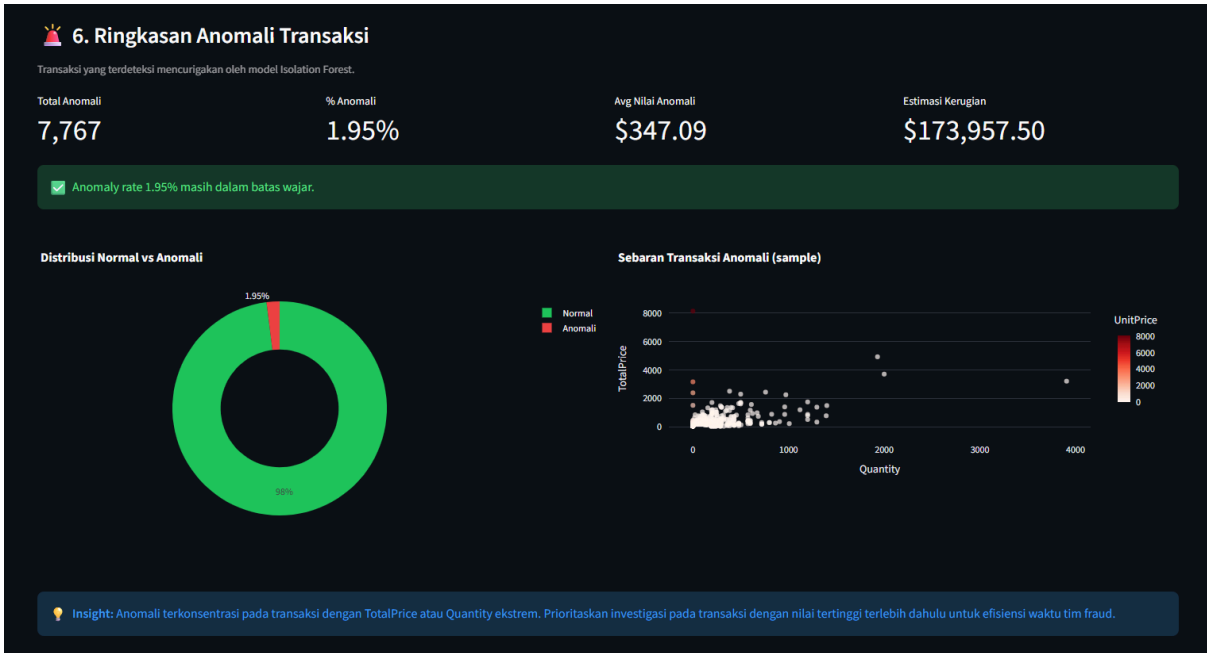
Dengan memanfaatkan data waktu transaksi, dashboard menampilkan visualisasi heatmap aktivitas pelanggan berdasarkan hari dan jam transaksi. Visualisasi ini membantu perusahaan dalam mengetahui waktu dengan tingkat aktivitas tertinggi, sehingga dapat digunakan

sebagai acuan dalam pengelolaan kapasitas server maupun penentuan jadwal promosi yang lebih efektif.



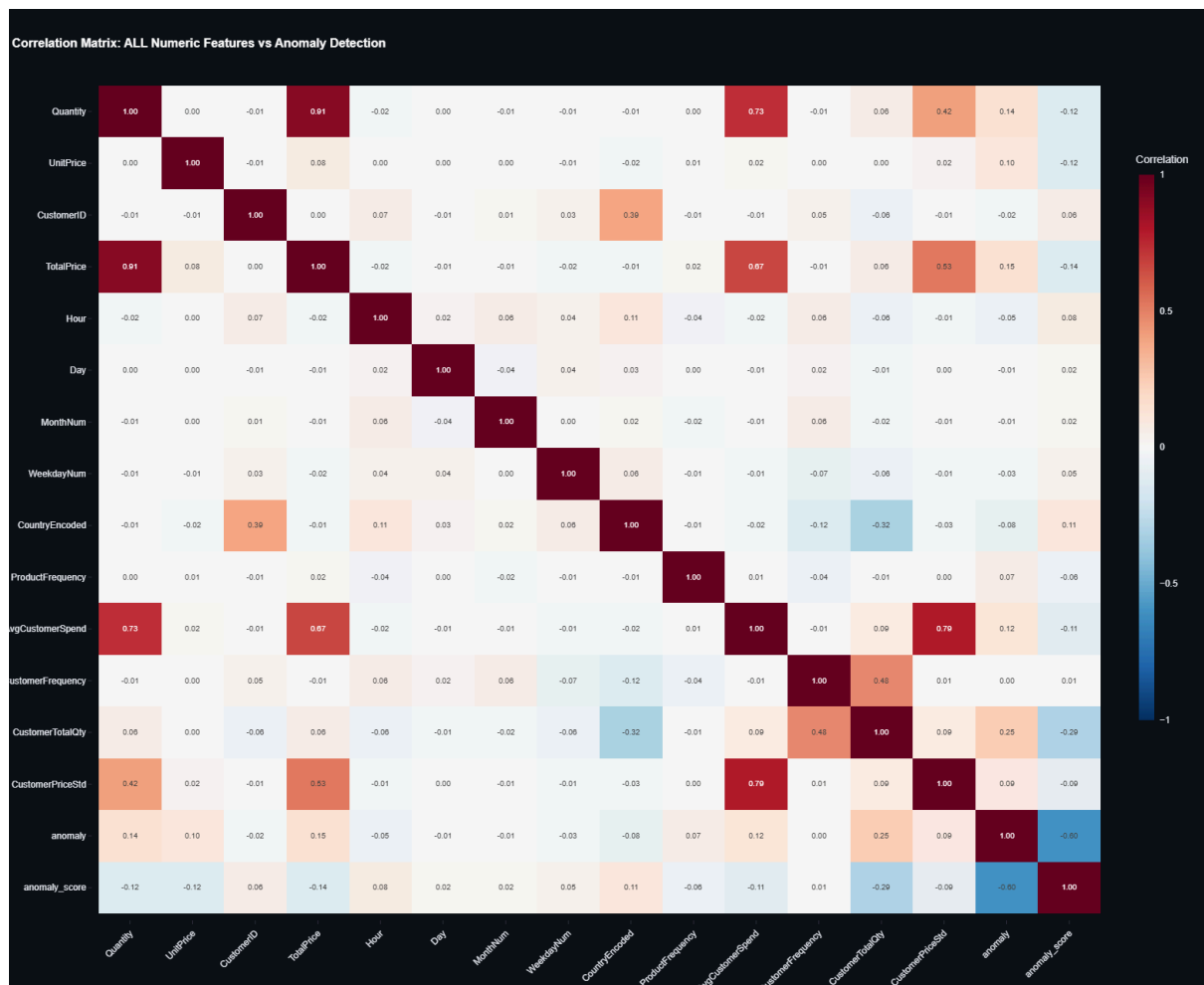
5.2.5 Ringkasan Anomali

Dashboard "Ringkasan Anomali Transaksi" ini berfungsi untuk mendeteksi dan menampilkan transaksi mencurigakan secara otomatis. Dengan memantau data ini, perusahaan bisa mendeteksi indikasi kecurangan (*fraud*) sejak dini dan menyelamatkan potensi kerugian keuangan yang nilainya mencapai ratusan ribu dolar. Saat ini, sistem mencatat ada 7.767 transaksi mencurigakan atau sekitar 1,95% dari total transaksi, yang artinya masih dalam batas aman (wajar). Melalui grafik lingkaran (*pie chart*) dan grafik sebaran (*scatter plot*) di atas, tim analis bisa melihat dengan jelas bahwa keanehan transaksi ini biasanya berkumpul pada pembelian produk dalam jumlah yang sangat banyak (*Quantity* ekstrem) atau transaksi dengan total belanja yang melonjak drastis (*TotalPrice* tinggi). Sebagai langkah tindak lanjut, data ini memberikan rekomendasi penting bagi tim keamanan untuk mendahulukan pemeriksaan pada transaksi-transaksi dengan nilai tertinggi terlebih dahulu.



5.2.6 Korelasi Fitur

Grafik matriks korelasi ini memberikan tiga *insight* penting terkait perilaku data transaksi dan sistem deteksi anomali. Terdapat hubungan positif yang sangat kuat (nilai 0.91) antara jumlah barang (*Quantity*) dan total harga (*TotalPrice*), yang menegaskan bahwa lonjakan kuantitas barang secara otomatis akan menaikkan nilai transaksi. Faktor waktu seperti jam, hari, dan bulan memiliki korelasi yang sangat lemah (mendekati 0) terhadap indikator *anomaly*, yang berarti aktivitas kecurangan dapat terjadi kapan saja secara acak tanpa terikat pada pola waktu tertentu. Nilai korelasi variabel tunggal terhadap *anomaly* yang cenderung kecil (di kisaran 0.10 hingga 0.25) membuktikan bahwa sistem deteksi kecurangan bekerja secara cerdas; sistem tidak hanya menilai transaksi dari satu faktor saja (seperti harga yang mahal saja), melainkan hasil dari kombinasi berbagai variabel perilaku belanja pelanggan yang tidak wajar.



BAB VI

KESIMPULAN

6.1 Kesimpulan

Berdasarkan pelaksanaan proyek ini, maka dihasilkan kesimpulan sebagai berikut:

1. Pipeline Big Data terbukti berhasil dijalankan secara berurutan, mulai dari pembersihan data mentah hingga penentuan status anomali transaksi.
2. Penggunaan algoritma Isolation Forest terbukti efektif dalam mendeteksi 7.767 transaksi yang mencurigakan dari keseluruhan 397.884 kumpulan data transaksi bersih.
3. Proses penyimpanan menggunakan metode ganda, yakni PostgreSQL (sebagai repositori utama) dan MongoDB (sebagai wadah penyimpanan dokumen khusus anomali) berjalan dengan baik dan saling melengkapi.
4. Tampilan dashboard visual cukup komprehensif untuk membantu tim manajemen memonitor pendapatan, produk, wilayah berisiko, dan anomali secara bersamaan.
5. Perbedaan jumlah temuan data di dashboard biasanya terjadi karena cache aplikasi belum memuat data pembaruan yang baru saja masuk ke basis data MongoDB.

6.2 Saran

Berikut adalah saran tindak lanjut berdasarkan kesimpulan dan hasil analisis proyek ini, dengan fokus pada pengembangan sistem dan optimalisasi operasional:

1. **Peningkatan Akurasi Model Deteksi Anomali Lanjutan.**
Disarankan untuk melakukan eksplorasi dan pengembangan lebih lanjut pada model deteksi anomali (Isolation Forest) dengan mengintegrasikan fitur-fitur perilaku pelanggan (AvgCustomerSpend, CustomerFrequency) yang telah dihasilkan dari proses *feature engineering*. Integrasi ini diharapkan dapat secara signifikan meningkatkan akurasi model dalam mengidentifikasi pola transaksi anomali yang lebih kompleks dan bernuansa.
2. **Optimalisasi Konsistensi Data pada Dashboard.**
Untuk mengatasi isu inkonsistensi jumlah data anomali yang disebabkan oleh *cache* aplikasi, perlu diimplementasikan mekanisme pembaruan data yang lebih sering, atau menerapkan sistem notifikasi *push* langsung dari basis data MongoDB ke *dashboard*. Langkah ini penting untuk memastikan data yang disajikan selalu mencerminkan kondisi terbaru dan akurat di database.
3. **Pemanfaatan Fleksibilitas MongoDB untuk Audit dan Pelatihan Ulang.**
Memaksimalkan keunggulan basis data non-relasional (MongoDB) dengan menyediakan akses bagi tim *Fraud Analyst* untuk menambahkan catatan investigasi terperinci pada setiap data anomali. Data yang diperkaya ini akan berfungsi sebagai materi audit yang komprehensif serta menjadi *dataset* berlabel yang kredibel untuk pelatihan ulang model di masa mendatang.

4. **Integrasi Wawasan Bisnis ke Dalam Strategi Operasional.**

Menggunakan temuan dari analisis bisnis yang telah dilakukan (Distribusi Geografis dan Pola Waktu/Jam Sibuk) sebagai dasar untuk pengambilan keputusan strategis yang lebih terarah, meliputi:

- Meningkatkan intensitas pengawasan tim *Fraud Analyst* pada wilayah geografis yang teridentifikasi memiliki tingkat anomali tinggi.
- Mengoptimalkan manajemen inventaris (penyediaan stok produk terlaris) dan perencanaan jadwal promosi agar selaras dengan jam transaksi tersibuk.

5. **Eksplorasi Pemrosesan *Near Real-Time (Stream Processing)*.**

Dalam rangka mendukung karakteristik *Velocity* pada *Big Data* (kecepatan produksi data), tim disarankan mempertimbangkan transisi arsitektur dari *batch processing* ke *stream processing* menggunakan teknologi seperti Apache Spark Streaming. Implementasi ini akan memungkinkan deteksi anomali dilakukan secara *near real-time*, sehingga respons dan mitigasi risiko bisnis dapat dilakukan lebih cepat.